H — Routing, cost-aware cascades and evaluation (the gate and the benchmark)

0. Role in the system

This section informs two modules. (1) **The gate**: for every subtask it decides between a *fast path* (skill library / 9B worker in VRAM) and a *slow path* (35B-A3B MoE senior in RAM + verified search), using similarity to already-solved tasks, a structural footprint of the task, and a self-estimate from the worker; its threshold is learned from logged outcomes under asymmetric error costs, and it is wrapped in a cheapest-first cascade that escalates when verification fails. (2) **The benchmark**: a personal, SWE-bench-style suite of 20 development tasks plus 10 held-out tasks, rotated monthly, graded by executing FAIL_TO_PASS / PASS_TO_PASS tests, with *skill-acquisition efficiency* (Chollet) rather than raw pass rate as the quantity tracked over months. After reading this pack an agent must be able to implement: the RouteLLM threshold router and its CPT/APGR evaluation; the FrugalGPT cascade with a scoring function and per-stage thresholds; a SWE-bench-compatible task record and grader; an SWE-agent-style tool interface for the worker; and the logging + rotation protocol that keeps the held-out set clean.

1. Papers

1.1 RouteLLM: Learning to Route LLMs with Preference Data (Ong et al., 2024) — arXiv:2406.18665

Problem and claim. Strong LLMs answer better but cost far more than weak ones (the paper cites Claude 3 Opus at $75 vs Claude 3 Haiku at $1.25 per million output tokens, a >50x gap). A learned *router* sends each query to either a strong or a weak model before generation. Trained on human preference data (Chatbot Arena) plus data augmentation, routers “reduce costs by over 2 times without sacrificing response quality” and transfer to unseen strong/weak pairs without retraining.

Headline results. Strong model `gpt-4-1106-preview`, weak model Mixtral-8x7B. Tables below are from the v1 HTML (Section 4 in v1; the same experiments are Section 5 in v4, whose HTML is truncated). Values are CPT(50%) / CPT(80%) / APGR (lower CPT is better; APGR change vs random in parentheses).

Benchmark	Training data	Best router	CPT(50%)	CPT(80%)	APGR
MT Bench (160 q)	Random	–	49.03%	78.08%	0.500
MT Bench	D_arena	SW ranking	37.85%	58.99%	0.610 (+22.1%)
MT Bench	D_arena + D_judge	Matrix factorization	13.40%	31.31%	0.802 (+60.4%)
MMLU 5-shot (14,042 q)	Random	–	50.07%	79.93%	0.500
MMLU	D_arena	Matrix factorization	45.00%	76.86%	0.524 (+4.9%)
MMLU	D_arena + D_gold	SW ranking	35.40%	71.55%	0.603 (+20.7%)

Benchmark	Training data	Best router	CPT(50%)	CPT(80%)	APGR
GSM8K 8-shot (>1,000 q)	Random	–	50.00%	80.08%	0.497
GSM8K	D_arena	SW ranking	54.43%	82.11%	0.4753 (-4.3%)
GSM8K	D_arena + D_judge	Causal LLM	33.64%	63.26%	0.622 (+25.3%)

Full rows (D_arena only → augmented): MT Bench BERT 78.09/87.64/0.391 → 19.58/34.02/0.751; Causal LLM 28.82/77.53/0.573 → 31.50/48.75/0.679; MF 25.32/74.26/0.580 → 13.40/31.31/0.802; SW 37.85/58.99/0.610 → 23.21/36.04/0.759. MMLU BERT 49.43/77.80/0.502 → 41.30/72.20/0.572; Causal LLM 48.88/77.93/0.499 → 35.49/70.31/0.600; MF 45.00/76.86/0.524 → 35.46/71.40/0.597; SW 55.82/80.25/0.473 → 35.40/71.55/0.603. GSM8K BERT 58.78/83.84/0.438 → 44.76/79.09/0.531; Causal LLM 56.09/83.56/0.461 → 33.64/63.26/0.622; MF 53.59/85.24/0.4746 → 38.82/72.62/0.565; SW 54.43/82.11/0.4753 → 41.21/72.20/0.568.

Cost saving over always-GPT-4 (Table 6, best routers): MT Bench 3.66x at CPT(50%) (95% of GPT-4 quality), 2.49x at CPT(80%); MMLU 1.41x (92% quality) / 1.14x; GSM8K 1.49x (87% quality) / 1.27x. Generalization without retraining to Claude 3 Opus (strong) / Llama 3 8B (weak) on MT Bench (Table 5): Random 47.23/77.08/0.494; D_arena+D_judge MF 30.48/41.81/0.703 (+42.2%); Causal LLM 40.46/55.83/0.625; SW 31.67/48.39/0.667; BERT 36.28/50.83/0.618. Router overhead (Table 7): SW ranking $37.36 per million requests (2.9 req/s), MF $1.42 (155.16 req/s), BERT $3.19 (69.62 req/s), Causal LLM $5.23 (42.46 req/s); at most 0.4% of GPT-4 generation cost.

Method — equations. (Section 3.1–3.2.) Models $M: \mathcal{Q} \to \mathcal{A}$; $\mathcal{M}_{\text{strong}}$, $\mathcal{M}_{\text{weak}}$ are the two classes; preference data $\mathcal{D}_{\text{pref}} = \{ \left(q, l_{s,w} \right) \}$ with $l_{s,w} \in \{\text{win}_s, \text{tie}, \text{win}_w\}$.

$$\max_{\theta} \sum_{\left(q, l_{s,w} \right) \in \mathcal{D}_{\text{pref}}} \log P_{\theta} \left(l_{s,w} \mid q \right) \quad \text{(Eq. 1)}$$

$$R^{\alpha}(q) = \begin{cases} \mathcal{M}_{\text{weak}} & \text{if } P\left(\text{win}_s \mid q \right) < \alpha \\ \mathcal{M}_{\text{strong}} & \text{if } P\left(\text{win}_s \mid q \right) \geq \alpha \end{cases} \quad \text{(Eq. 2)}$$

$\alpha \in [0,1]$ is the cost threshold; higher α sends more queries to the weak model. The routed model is $M_{R^{\alpha}}\left(q, M_s, M_w \right) = M_s(q)$ if $R^{\alpha}(q) = \mathcal{M}_{\text{strong}}$, else $M_w(q)$ (Eq. 3).

$$c\left(M_{R^{\alpha}} \right) = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \mathbb{I}\{R^{\alpha}(q) = \mathcal{M}_{\text{strong}}\} \quad \text{(Eq. 4)}$$

$$r\left(M_{R^{\alpha}} \right) = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} s(M_{R^{\alpha}}(q)) \quad \text{(Eq. 5)}$$

$s(\cdot)$ is a per-response score (correctness on a golden-labelled benchmark, or a judge rating).

$$PGR\left(M_{R^{\alpha}} \right) = \frac{r\left(M_{R^{\alpha}} \right) - r\left(M_w \right)}{r\left(M_s \right) - r\left(M_w \right)} \quad \text{(Eq. 6)}$$

$$APGR\left(M_{R^{\alpha}} \right) = \int_0^1 PGR\left(M_{R^{\alpha}} \right) d(c\left(M_{R^{\alpha}} \right)) \approx \frac{1}{10} \sum_{i=1}^{10} PGR(M_{R^{\alpha_i}}) \quad \text{(Eqs. 7–8)}$$

where α_i are chosen so that call fractions c_i spread over $[0\%, 100\%]$ at 10 points. **CPT(x%)** (defined in prose, not numbered): the minimum fraction of calls to the strong model needed to reach PGR $= x\%$; e.g. CPT(50%) $\approx 37\%$ in Figure 1. Benchmark–dataset similarity (Eq. 13, Appendix C): mean over evaluation prompts of the *maximum* cosine similarity to any training prompt; Table 4 reports MT Bench 0.6078 (Arena) / 0.6525 (augmented), MMLU 0.4823 / 0.5678, GSM8K 0.4926 / 0.5335, and higher similarity correlates with better routing.

Method — algorithm. Four routers, all estimating $P(\text{win}_s \mid q)$: 1. **Similarity-weighted (SW) ranking.** Embed q and all training queries q_i. Weight $\omega_i = \gamma^{1+S(q,q_i)}$ with $\gamma = 10$, where S is cosine similarity divided by the largest cosine similarity between q_i and any training embedding. Fit a Bradley–Terry model with coefficients ξ_s, ξ_w by minimising $\sum_i \omega_i \cdot \text{BCE}\left(l_{s,w}^{(i)}, P(\text{win}_s \mid q_i)\right)$, with $P(\text{win}_s \mid q) = 1/\left(1 + e^{\xi_s - \xi_w}\right)$ (sign as rendered in the fetched HTML — verify §4.1). No training; solved at inference per query. 2. **Matrix factorization.** Score $\delta(M, q) = w_2^T(v_m \odot \left(W_1^T v_q + b\right))$ with model embedding v_m, query embedding v_q, projection W_1, b, output vector w_2; $P(\text{win}_s \mid q) = \sigma\left(\delta(M_s, q) - \delta(M_w, q)\right)$; trained by Eq. 1. 3. **BERT classifier.** BERT-base, $P(\text{win}_s \mid q) = \sigma\left(Wh_{\text{CLS}} + b\right)$; full fine-tuning. 4. **Causal LLM classifier.** Llama 3 8B, instruction prompt containing q; comparison labels added as new vocabulary tokens; $P(\text{win}_s \mid q)$ = softmax over the label tokens.

Hyperparameters and setup. Data: 80K Chatbot Arena battles → prompts <16 characters pruned → ~65K pairwise comparisons across 64 models; 5K held out for validation; 81% English. Label sparsity (<0.1% per pair) handled by grouping models into 10 tiers via dynamic programming (top two tiers = strong, third tier = weak). Augmentation: D_gold ≈ 1,500 MMLU validation questions (<2% of training data); D_judge ≈ 120K Nectar prompts, GPT-4 responses as strong, Mixtral-8x7B as weak, GPT-4 judge, ≈$700. Embeddings: `text-embedding-3-small` (MF and SW). MF: ~10 epochs, batch 64, Adam, lr 3×10^{-4}, weight decay 10^{-5}, 8 GB GPU. BERT: ~2,000 steps, batch 16, max len 512, lr 10^{-5}, weight decay 0.01, 2x L4 24GB. Causal LLM: ~2,000 steps, batch 8, max len 2048, lr 10^{-6}, 8x A100 80GB. Contamination check: embedding similarity ≥ 0.95 between eval and training items flagged and discarded. Threshold calibration (repo): `python -m routellm.calibrate_threshold --task calibrate --routers mf --strong-model-pct 0.5` picks α so that a target fraction of a sample of the user's own queries goes to the strong model.

Ablations that matter. Arena-only training is near random on MMLU and GSM8K (APGR 0.47–0.52); adding ~1,500 in-domain golden labels lifts MMLU APGR to 0.60 (+20.7%); adding judge labels lifts MT Bench MF APGR from 0.580 to 0.802 and GSM8K Causal LLM from 0.461 to 0.622. Routers trained on one pair transfer to Claude 3 Opus / Llama 3 8B with comparable APGR (0.703 vs 0.802). Similarity between eval and training prompts predicts router success.

Code / data / license. https://github.com/lm-sys/RouteLLM (Apache-2.0); routers and data under the RouteLLM / LMSYS Hugging Face organisations; default calibration set `lmsys/lmsys-arena-human-preference-55k`.

Stated limitations. Real query distributions differ from benchmarks (small in-domain data helps); only two-model routing; router performance varies widely on identical data without explanation; sparse human labels limit parameter-heavy routers; judge labels carry bias.

Use in our design. The gate is exactly R^α with $P(\text{win}_s \mid q)$ replaced by $P(\text{fast path yields a verified pass} \mid q)$. Take the SW-ranking idea (weighted neighbours among *our own* solved tasks, $\gamma = 10$) as the first feature and the MF/logistic head as the learned combiner; measure the gate with PGR/APGR/CPT where cost is the fraction of senior calls. Replace OpenAI embeddings with a local embedding model; replace Arena labels with logged verified outcomes; recalibrate α monthly on recent tasks (the repo's calibrate step). PDF: https://arxiv.org/pdf/2406.18665 — Code: https://github.com/lm-sys/RouteLLM

1.2 FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance (Chen et al., 2023) — arXiv:2305.05176

Problem and claim. LLM API prices differ by up to two orders of magnitude (Table 1, March 2023, per 10M tokens: GPT-J $0.2 input / $5 output; GPT-4 $30 / $60; GPT-3 $20 / $20; ChatGPT $2 / $2; J1-Jumbo $0 / $250 + $0.005 per request; Cohere Xlarge $10 / $10). Three strategies cut cost: prompt adaptation, LLM approximation, and the LLM cascade. FrugalGPT (the cascade) learns which APIs to call in which order with per-stage acceptance thresholds, and "can match the performance of the

best individual LLM (e.g. GPT-4) with up to 98% cost reduction or improve the accuracy over GPT-4 by 4% with the same cost."

Headline results. Table 3 (cost to match the best single LLM): HEADLINES — GPT-4 33.1 vs FrugalGPT 0.6, 98.3% saving; OVERRULING — 9.7 vs 2.6, 73.3%; COQA — best single is GPT-3 at 72.5 vs 29.6, 59.2% (cost units not stated in text). HEADLINES case study: budget $6.5 (one fifth of GPT-4's cost) gives 80% cost reduction and +1.5% accuracy over GPT-4. OVERRULING: +1% accuracy at 73% lower cost. Accuracy gains "up to 4%" (abstract) / "up to 5%" (Figure 5 caption) at equal cost.

Method — equations. (Section 2, inline.) Cost of API i on prompt p:

$$c_i(p) \triangleq \tilde{c}_{i,2} \parallel f_i(p) \parallel + \tilde{c}_{i,1} \parallel p \parallel + \tilde{c}_{i,0}$$

$f_i: \mathcal{P} \to \mathcal{A}$ is API i among K; $\parallel p \parallel$ prompt length, $\parallel f_i(p) \parallel$ generation length; $\tilde{c}_{i,2}, \tilde{c}_{i,1}, \tilde{c}_{i,0}$ per-output-token, per-input-token and per-request constants. Scoring function $g(\cdot, \cdot): \mathcal{Q} \times \mathcal{A} \to [0,1]$ (reliability of an answer). Cascade optimisation (Section 3, unnumbered display):

$$\begin{aligned} \max_{L,\tau} \quad & \mathbb{E}[r\left(a, f_{L_z}(q)\right)] \\ \text{s.t.} \quad & \mathbb{E}[\textstyle\sum_{i=1}^{z} \tilde{c}_{L_i,2} \parallel f_{L_i}(q) \parallel + \tilde{c}_{L_i,1} \parallel q \parallel + \tilde{c}_{L_i,0}] \leq b, \\ & z = \arg\min_i \{i : g\left(q, f_{L_i}(q)\right) \geq \tau_i\} \end{aligned}$$

$L \in [K]^m$ is the ordered list of m APIs; τ_i the acceptance threshold at position i; z the first position whose score passes its threshold (the answer returned); $r\left(a, \widehat{a}\right)$ the reward of the answer against the correct a; b the average budget per query.

Method — algorithm. 1. *Prompt adaptation.* (a) Prompt selection: keep only a small subset of in-context examples. (b) Query concatenation: pack several queries into one prompt with instructions to answer all. No numeric settings given. 2. *LLM approximation.* (a) Completion cache: store (query, response) locally; on a new query, look up a similar cached query and return its response; call the API only on a miss. (b) Model fine-tuning: collect the expensive LLM's responses to a few queries, fine-tune a cheaper model on them, serve the cheap model (also reduces latency because long prompts are no longer needed). 3. *LLM cascade.* For $i = 1..m$: query $f_{L_i}(q)$; compute $g\left(q, f_{L_i}(q)\right)$; if $\geq \tau_i$ return the answer, else continue. The *router* learns (L, τ) jointly by solving the problem above; it is mixed-integer, so the authors (a) prune candidate lists with small answer disagreement between APIs and (b) approximate the objective by interpolating from a few samples. Learned HEADLINES cascade: GPT-J → J1-L → GPT-4 with thresholds 0.96 (accept GPT-J) and 0.37 (accept J1-L). 4. *Scoring function.* A small regression model that predicts whether a generation is correct given (query, answer); the case study uses DistilBERT tailored to regression, "considerably smaller and therefore less expensive than all LLMs considered."

Hyperparameters and setup. 12 APIs from 5 providers (OpenAI, AI21, Cohere, Textsynth, ForeFrontAI); cascade length 3. Datasets (Table 2): HEADLINES (finance, 10,000 items, 8 in-prompt examples, gold-price trend up/down/neutral/none), OVERRULING (law, 2,400, 5 examples, is-a-sentence-an-overruling), COQA (reading comprehension, 7,982, 2 examples, adapted to direct QA). Each dataset randomly split into train (learn cascade) and test; split ratio, scorer training size and DistilBERT settings not in fetched text (verify in paper §4).

Ablations that matter. None reported as ablations. The key evidence is the cost–accuracy frontier (Figure 5) and the observation that cheap models' answers, when the scorer accepts them, are at least as reliable as GPT-4's on the same items.

Code / data / license. https://github.com/stanford-futuredata/FrugalGPT (Apache-2.0; HEADLINES data and a `qa_cache.sqlite` completion cache downloaded at install; AGNEWS and SCIQ notebooks in the changelog). The paper itself gives no link.

Stated limitations. Needs labelled examples from the same or a similar distribution; learning the cascade is an upfront cost; latency, fairness, privacy and environmental impact not addressed; risk-

critical use needs uncertainty quantification; when all chain members agree the cascade may still call all of them because it cannot tell whether the first answer is correct (open problem).

Use in our design. Our verifier cascade *is* a FrugalGPT cascade with L = [skill, worker, senior+search] and g = the verifier result (tests, lint, type-check) where tests exist, falling back to a learned g (small classifier on logged outcomes) when they do not. The completion cache becomes the skill library + Notebook; "model fine-tuning on strong outputs" is the weekly senior→worker distillation. Replace API dollars with seconds and VRAM/RAM occupancy in c_i; the budget b is the user's latency tolerance per task class. PDF: https://arxiv.org/pdf/2305.05176 — Code: https://github.com/stanford-futuredata/FrugalGPT

1.3 SWE-bench: Can Language Models Resolve Real-World GitHub Issues? (Jimenez et al., 2024) — arXiv:2310.06770

Problem and claim. Models have outpaced static benchmarks. Real GitHub issues give a renewable, execution-graded testbed requiring multi-file edits, long context and environment interaction. SWE-bench has 2,294 task instances from 12 Python repositories; the best model at publication (Claude 2) resolves 1.96% (abstract; Table 5 lists 1.97%).

Headline results. % Resolved / % Apply on full SWE-bench with BM25 retrieval (Table 5, v3): Claude 3 Opus 3.79 / 46.56; Claude 2 1.97 / 43.07; GPT-4-turbo 1.31 / 26.90; ChatGPT-3.5 0.17 / 26.33; SWE-Llama 7b 0.70 / 51.74; SWE-Llama 13b 0.70 / 53.62. SWE-bench Lite (300 instances, 11 repos, BM25): Claude 3 Opus 4.33 / 51.67; Claude 2 3.00 / 33.00; GPT-4-turbo 2.67 / 29.67; ChatGPT-3.5 0.33 / 10.00; SWE-Llama 7b 1.33 / 38.00; 13b 1.00 / 38.00. Oracle retrieval (gold-patch files given): Claude 2 4.8%, GPT-4 1.3% (GPT-4 on a 25% subset); Claude 2 resolves 110 instances and SWE-Llama 13b 91, overlapping only 42%. Oracle-collapsed (non-edited code collapsed, ±15-line buffer): Claude 3 Opus 9.39 / 48.00; Claude 2 5.93 / 68.18; GPT-4 3.40 / 48.65; ChatGPT-3.5 1.09 / 40.93. BM25 recall of oracle files (Table 3, avg / all / any): 13k tokens 29.58 / 26.09 / 34.77; 27k 44.41 / 39.83 / 51.27; 50k 51.06 / 45.90 / 58.38 — yet resolve rates *fall* with more context (Figure 5). Temporal split (Table 7, pre- vs post-2023 PRs): Claude 2 4.87 vs 4.23; SWE-Llama 7b 2.95 vs 3.46; 13b 3.98 vs 3.85; GPT-4 1.96 vs 0.0 (25% subset, cautioned).

Method — equations. No equations; the benchmark's metric is the fraction of instances resolved:

$$\text{Resolved}\left(\widehat{\delta}\right) = \mathbb{I}[\text{apply}\left(\widehat{\delta}\right) \wedge \forall t \in \text{F2P} \cup \text{P2P}: \text{pass}\,(\,t\,)\,]$$

where $\widehat{\delta}$ is the generated patch, F2P the FAIL_TO_PASS list, P2P the PASS_TO_PASS list (formalisation of the prose rule "the patch applies and all associated tests pass").

Method — algorithm. Construction (Section 2.1): 1. *Stage I — scrape.* Pull requests from 12 popular Python repositories (~90,000 PRs), each tied to a base commit. 2. *Stage II — attribute filter.* Keep merged PRs that (a) resolve a GitHub issue and (b) change test files. 3. *Stage III — execution filter.* Apply the PR's test changes; run tests before and after the PR's non-test changes; keep instances with ≥1 FAIL_TO_PASS test; drop installation/runtime errors → 2,294 instances. 4. *Definitions.* **FAIL_TO_PASS**: tests whose status flips fail→pass when the reference non-test changes are applied (≥1 per instance; ~40% have ≥2). **PASS_TO_PASS**: tests that pass both before and after the PR (median 51 per instance); they check existing functionality is maintained. 5. *Task.* Input: issue text + full codebase snapshot. Output: a patch file. *Evaluation.* Apply the patch with Unix `patch`; run the instance's tests; resolved only if it applies and all tests pass. Log-parsing details are in Appendix A.4 (not in fetched text). 6. *Dataset statistics (Table 1, mean / max).* Issue length 195.1 / 4,477 words; codebase 3,010 / 5,890 non-test files, 438K / 886K lines; gold patch 32.8 / 5,888 lines, 1.7 / 31 files, 3 / 36 functions; FAIL_TO_PASS 9.1 / 1,633; total tests 120.8 / 9,459. 7. *Instance schema (HF dataset card).* `instance_id`, `repo`, `base_commit`, `patch` (gold, non-test), `test_patch`, `problem_statement` (issue title + body), `hints_text` (issue comments before the first PR commit), `created_at`, `version`, `environment_setup_commit`, `FAIL_TO_PASS` (JSON list), `PASS_TO_PASS` (JSON list). Splits: train ≈19K, dev 225, test 2,294.

Hyperparameters and setup. BM25 context limits 13k / 27k / 50k tokens. SWE-Llama: CodeLlama-Python 7B and 13B; SWE-bench-train ≈19,000 issue-PR instances from 37 repositories disjoint from the 12 evaluation repos (test changes not required); sequences >30k tokens dropped → ≈10,000; LoRA on attention sublayers only; rank, lr, epochs, hardware in Appendix B (not in fetched text — verify). Generated patches are shorter than gold: Claude 2 19.6 vs 44.1 changed lines, SWE-Llama 13b 17.6 vs 37.8 (Table 8), and rarely touch >1 file.

Ablations that matter. Context helps retrieval recall but hurts resolution (localisation failure). Oracle-collapsed more than doubles GPT-4 (1.3→3.4) and lifts Claude 2 (4.8→5.9). Little pre/post-2023 difference → low memorisation. Fine-tuned SWE-Llama learned to edit every file in context and so fails under BM25 context.

Code / data / license. swebench.com; https://github.com/SWE-bench/SWE-bench (MIT); Hugging Face `princeton-nlp/SWE-bench`. Harness: `swebench eval verified -p <predictions> --run-id <id> -j <workers>` (older `python -m swebench.harness.run_evaluation`), Docker images per instance, results in `logs/evaluation/<run_id>/results.json`, cached by `run_id` + `instance_id`; recommends x86_64, ≥120 GB disk, 16 GB RAM, 8 cores.

SWE-bench Verified (OpenAI post, secondary source; not in the paper). 500-instance human-validated subset that supersedes full and Lite. 93 Python-experienced developers annotated 1,699 random test items, 3 annotators each, highest severity kept; two 0–3 severity criteria — issue underspecification and FAIL_TO_PASS tests unfairly rejecting valid solutions — plus a time-to-fix estimate and a free-form major-issue flag; items with severity ≥2 on either criterion or any major flag discarded (38.3% flagged underspecified, 61.1% unfair tests, 68.3% filtered overall); the kept 500 prioritise 1–4 h and >4 h estimates, rest random; easy subset 196 tasks <15 min, hard 45 tasks >1 h. GPT-4o (Agentless) 33.2% on Verified vs 16% on original. Harness containerised in Docker.

Stated limitations. Python only; test-based evaluation "is insufficient to guarantee reliable performance" (readability, efficiency not graded); some issues embed images (32% of matplotlib, 10% of seaborn instances vs 2% overall); oracle retrieval unrealistic; baselines simple. Lite construction criteria in Appendix A.7 and the per-repository counts in Figure 3 are not in the fetched text.

Use in our design. The personal benchmark copies the instance schema and the three-stage construction verbatim from the user's own repositories (merged commits that touch tests → execute before/after → keep those with ≥1 F2P); the grader is the Resolved rule above run in a container. Verified's annotation rubric (underspecification 0–3, unfair-test 0–3, discard at ≥2) is the triage the user applies when a mined task is noisy. The temporal-split argument justifies always mining held-out tasks newer than the last LoRA training cut. PDF: https://arxiv.org/pdf/2310.06770 — Code: https://github.com/SWE-bench/SWE-bench

1.4 On the Measure of Intelligence (Chollet, 2019) — arXiv:1911.01547

Problem and claim. Task-specific skill is not intelligence, because skill can be bought with priors (built-in knowledge) or experience (training data). Intelligence should be measured as *skill-acquisition efficiency* over a scope of tasks, controlling for priors, experience and generalisation difficulty. The paper gives an Algorithmic-Information-Theory definition and derives the Abstraction and Reasoning Corpus (ARC).

Headline results. Conceptual paper; no benchmark numbers. ARC: 400 training tasks and 600 evaluation tasks (400 public + 200 private, per secondary sources — the fetched HTML stops before Section III); each task has demonstration input/output grid pairs (typically 3) and test inputs (typically 1); grids 1x1 to 30x30, 10 colours; 3 trials per test input; success is an exact cell-for-cell match (ARC-AGI repository, Apache-2.0).

Method — equations. (Section II.2; symbols from the fetched text.) $H(s)$ = length of the shortest program producing s on a fixed universal Turing machine; $H(s_1 \mid s_2)$ = shortest program producing s_1 given s_2 as free input. Sol_T^{θ} = shortest skill program reaching skill $\geq \theta$ on task T; $TrainSol_{T,C}^{opt}$ =

shortest program with optimal training-time performance on curriculum C; IS_t = the intelligent system's state at step t; $data_t$ = situation, response and feedback at step t.

$$GD^{\theta}_{T,C} = \frac{H(Sol^{\theta}_{T} \mid TrainSol^{opt}_{T,C})}{H(Sol^{\theta}_{T})}$$

$$GD^{\theta}_{IS,T,C} = \frac{H(Sol^{\theta}_{T} \mid TrainSol^{opt}_{T,C},\, IS_{t=0})}{H(Sol^{\theta}_{T})} \quad \text{(developer-aware generalisation difficulty)}$$

$$P^{\theta}_{IS,T} = \frac{H(Sol^{\theta}_{T}) - H(Sol^{\theta}_{T} \mid IS_{t=0})}{H(Sol^{\theta}_{T})} \quad \text{(priors)}$$

$$E^{\theta}_{IS,T,t} = H(Sol^{\theta}_{T} \mid IS_t) - H(Sol^{\theta}_{T} \mid IS_t, data_t), \qquad E^{\theta}_{IS,T,C} = \frac{1}{H\left(Sol^{\theta}_{T}\right)} \sum_{t} E^{\theta}_{IS,T,t} \quad \text{(experience)}$$

$$I^{\theta_T}_{IS,\,scope} = \underset{T \in scope}{\text{Avg}} \left[\omega_T \cdot \theta_T \sum_{C \in Cur^{\theta_T}_{T}} P_C \cdot \frac{GD^{\theta_T}_{IS,T,C}}{P^{\theta_T}_{IS,T} + E^{\theta_T}_{IS,T,C}} \right]$$

ω_T = value of task T; θ_T = sufficient-skill threshold; $Cur^{\theta_T}_{T}$ = set of curricula leading IS to a sufficient solution; P_C = probability of curriculum C. Reading: intelligence is generalisation difficulty achieved per unit of (priors + experience) spent, averaged over valued tasks. Skill itself is the output artefact, not the measure.

Method — algorithm. Not an algorithm paper. The benchmark guidelines (Section II.3.2, from a secondary summary because the HTML truncates at II.3.1): state the scope and the benchmark's own predictiveness for that scope; be reproducible; measure broad abilities and developer-aware generalisation, not skill; control the amount of experience the system can accrue; declare all assumed priors; assume only human Core Knowledge priors and human-sized practice. ARC's Core Knowledge priors (Section III.1.2): *objectness* (cohesion, persistence under noise/occlusion, influence via contact); *goal-directedness* (pairs as start/end states of an intentional process; no explicit time); *numbers and counting* (count, sort by size, compare, repeat N times); *basic geometry and topology* (lines, rectangles, symmetry, rotation, translation, scaling, distortion, containment, drawing/connecting, projection, copying). Stated ARC weaknesses (Section III.3): generalisation not quantified; test validity not established; dataset size/diversity limited; evaluation close-ended and binary; Core Knowledge priors possibly poorly captured.

Hyperparameters and setup. None beyond the ARC format above. Evaluation tasks are unique and unknown to developers (developer-aware generalisation).

Ablations that matter. None.

Code / data / license. https://github.com/fchollet/ARC-AGI (Apache-2.0; JSON tasks with `train` and `test` lists of `{input, output}` grids).

Stated limitations. H is uncomputable, so the definition is a guide to what to control for, not a formula to evaluate numerically; the author lists ARC's weaknesses above.

Use in our design. The benchmark tracks *efficiency*, not pass rate: for each held-out task measure the experience spent (attempts, tokens, wall-clock, senior calls) to reach a verified pass, with priors fixed by freezing the base model and difficulty approximated by novelty (distance to nearest solved task, cf. GD conditioning on what the system already holds). "Developer-aware generalisation" is why the 10 held-out tasks must never leak into the skill library, Notebook or LoRA data. The declared-priors rule becomes: record exactly which skills, memories and LoRA pockets were loaded when a task was attempted. PDF: https://arxiv.org/pdf/1911.01547 — Code: https://github.com/fchollet/ARC-AGI

1.5 SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering (Yang et al., 2024) — arXiv:2405.15793

Problem and claim. LM agents in a raw Linux shell make small file edits badly and get no feedback on invalid ones. Agents are a new kind of end user and need an *agent–computer interface* (ACI): few simple actions, compact feedback and guardrails. Without changing weights, the ACI alone lifts SWE-bench resolution from 3.8% (previous best non-interactive) to 12.47%.

Headline results. Table 1 (% Resolved / $ average cost): SWE-bench full (2,294): RAG w/ GPT-4 Turbo 1.31 / 0.13; RAG w/ Claude 3 Opus 3.79 / 0.25; SWE-agent w/ GPT-4 Turbo **12.47** (286/2294) / 1.59; SWE-agent w/ Claude 3 Opus 10.46 / 2.59. SWE-bench Lite (300): RAG GPT-4 Turbo 2.67 / 0.13; RAG Opus 4.33 / 0.25; Shell-only agent w/ GPT-4 Turbo 11.00 / 1.46; Shell-only w/o demonstration 7.33 / 0.79; SWE-agent GPT-4 Turbo **18.00** / 1.67; SWE-agent Opus 13.00 / 2.18. HumanEvalFix pass@1 (Table 2): Python 87.7, JavaScript 89.7, Java 87.9 (results text says 88.3 for Python — inconsistent). pass@k over 6 runs on Lite shown in Figure 4; a pass@6 number is not in the fetched text (verify Appendix B.5).

Method — equations. None. Metric: % Resolved = pass@1 = "the proportion of instances for which all tests pass successfully after the model generated patch is applied to the repository."

Method — algorithm. ACI principles (Section 2): (1) simple actions with few options and concise documentation; (2) compact, efficient actions — consolidate navigation and editing; (3) informative but concise feedback on state and effects; (4) guardrails — automatic checks (syntax) that catch mistakes early. Interface (Section 3): 1. *Search/navigation:* `find_file` (filename in repo), `search_file` (string in file), `search_dir` (string across directory); each returns ≤50 results, otherwise results are withheld and the agent is told to narrow the query. 2. *File viewer:* `open` shows a window of ≤100 lines with full path, total line count, lines omitted above/below, and line numbers; `scroll_up` / `scroll_down` move by 100 lines; `goto <line>`. 3. *File editor:* `edit <start_line>:<end_line>` + replacement text replaces the range in one step and re-shows the window; a `flake8` lint runs on the result and if the edit introduces a syntax-type error the edit is discarded and the agent sees the error, the would-be snippet and the original, and is asked to retry. 4. *Context management:* ReAct-style thought + action per turn; malformed responses get an error and, once a valid response arrives, all but the first past error messages are removed; empty command output is replaced by "ran successfully and did not produce any output"; observations older than the last 5 are collapsed to a single line; the agent receives instructions, command docs and an optional demonstration. 5. `submit`: produce the patch from all edits and close the shell. Budget: $4 per instance; on exhaustion existing edits are auto-submitted (≈30–40 turns).

Hyperparameters and setup. `gpt-4-1106-preview` (128k context) and `claude-3-opus-20240229` (200k); temperature 0.0; window 100 lines; history last 5 observations; chosen by a sweep on 37 dev-split instances, mean over 5 samples. Behaviour: resolved GPT-4 Turbo runs average 14.71 turns (median 12; 75% ≤18); Opus on Lite 12.71 (median 13); successful runs median $1.21 and 12 steps vs unsuccessful mean $2.52 and 21 steps; 93.0% of resolved instances submitted before budget exhaustion vs 69.0% overall; turns 1–3 dominated by reproduction and localisation (`create, edit, python` the most common opening, 156 occurrences); from turn ~5, `edit` / `python` alternate; submissions peak turns 10–20.

Ablations that matter. (Table 3, Lite, GPT-4 Turbo, baseline 18.0%.) Edit: w/o linting 15.0 (-3.0); no edit command (shell editing) 10.3 (-7.7). Viewer window: 30 lines 14.3 (-3.7); full file 12.7 (-5.3). Search: iterative 12.0 (-6.0); no search tools 15.7 (-2.3). Context: full history 15.0 (-3.0); no demonstration 16.3 (-1.7). Editing failures: 51.7% of instances have ≥1 failed edit (185 instances; denominator garbled in the fetched HTML); 31.5% of resolved instances do (113); an edit succeeds 90.5% of the time with no prior failure, 57.2% after one failure. Failure taxonomy (248 unresolved Lite instances, GPT-4o-labelled, 87% agreement with authors): 52.0% incorrect or overly specific implementation; 23.4% cascading failed edits.

Code / data / license. swe-agent.com; https://github.com/SWE-agent/SWE-agent (MIT; YAML-configured tools; maintainers now recommend mini-SWE-agent). Paper licence CC BY 4.0.

Stated limitations. Agents re-edit the same snippet after syntax errors; iterative search exhausts budget/context; later turns are used poorly (local edits rather than alternatives); 52% of failures are logic errors the ACI cannot fix; Limitations section E.3 not in fetched text.

Use in our design. The worker's tool surface should be the SWE-agent ACI (≤50 search hits, 100-line window, range edit with lint-reject, last-5-observation history, `submit`), which is cheap to implement locally and gives the gate structural signals for free: number of search calls, files opened, failed edits and turns are footprint features and early-escalation triggers (e.g. escalate to the senior after the second failed edit, since recovery drops to 57.2%). PDF: https://arxiv.org/pdf/2405.15793 — Code: https://github.com/SWE-agent/SWE-agent

2. Design extraction for the module

Everything in this section is **design proposed for our system**, derived from the papers above; it is not a paper result unless a citation says so.

2.1 Algorithms to implement (consolidated pseudocode)

A. The gate (RouteLLM R^{α} + FrugalGPT cascade).

```
function ROUTE_AND_SOLVE(task):
    x      = GATE_FEATURES(task)                    # §2.3 GateFeatures
    p_fast = CALIBRATED_P(x)                        # P(fast path yields verified pass | x)
    alpha  = CURRENT_THRESHOLD()                    # learned, §B
    if task.must_be_fast or p_fast >= alpha:        # Eq. 2 with roles swapped: fast if
confident
        decision = FAST
    else:
        decision = SLOW
    LOG(RouteDecision{task, x, p_fast, alpha, decision})
    # cascade L = [skill, worker, senior+search]; stop when verifier accepts (FrugalGPT z)
    stages = [SKILL, WORKER, SENIOR] if decision == FAST else [SENIOR]
    for i, stage in enumerate(stages):
        a   = stage.run(task, budget_i)
        g   = VERIFY(task, a)                        # 1.0 if tests/lint/typecheck pass; learned
g in [0,1] if no tests
        LOG(StageOutcome{task, stage, g, cost=seconds, tokens, failed_edits, turns})
        if g >= tau[stage]: return a, i
        if stage == WORKER and a.failed_edits >= 2: break   # SWE-agent: recovery falls to 57.2%
after one failure
    return ESCALATE_TO_USER(task)
```

B. Learning the threshold from logged outcomes with asymmetric costs.

Let C_f = expected cost of a fast attempt, C_s = expected cost of a slow attempt, λ = extra penalty for a *false-fast* (a failed fast attempt that is user-visible and then escalates). Expected cost of routing fast $= C_f + (1-p)\,(C_s+\lambda)$; of routing slow $= C_s$. Route fast iff

$$p \geq \alpha^{*} = \frac{C_f + \lambda}{C_s + \lambda}$$

```
function RELEARN_THRESHOLD(log, window=last 30 days):
    D = [(x_i, y_i)] from log where y_i = 1 if the FAST stages produced a verified pass
    P = fit LogisticRegression(x -> y)  then  isotonic calibration on a held-out 20% of D
    C_f, C_s = median wall-clock seconds of FAST and SLOW outcomes in D
    lambda   = user setting (default 2 * C_s; a wasted fast attempt is worse than a slow one)
    alpha_cost = (C_f + lambda) / (C_s + lambda)
    # RouteLLM calibrate_threshold: optionally cap senior usage at a budget fraction b
    alpha_budget = quantile of P(x_i) such that frac(P(x_i) < alpha) == b
    alpha = max(alpha_cost, alpha_budget)            # the stricter of the two
    report CPT(80%), APGR over the sweep (§2.5); store alpha with a version
```

Cold start (fewer than ~50 logged outcomes): use the SW-ranking rule from RouteLLM with our own solved tasks as the preference set — $p = \sum_i \omega_i y_i / \sum_i \omega_i$, $\omega_i = \gamma^{1+S(x,x_i)}$, $\gamma = 10$ — and $\alpha = 0.7$.

C. Benchmark task mining (SWE-bench three-stage construction, local).

```
function MINE_TASKS(repo, since_commit):
    for commit in merged commits after since_commit that touch test files:
        base = commit.parent
        split commit diff into test_patch (test files) and patch (the rest)
        checkout base; apply test_patch; run tests -> status_before
        apply patch; run tests -> status_after
        F2P = {t : status_before[t]==FAIL and status_after[t]==PASS}
        P2P = {t : status_before[t]==PASS and status_after[t]==PASS}
        if |F2P| >= 1 and no install/runtime errors:
            emit BenchmarkTask{instance_id, repo, base_commit=base, patch, test_patch,
                               problem_statement=commit message + linked issue, F2P, P2P,
created_at}
    # Verified-style triage: user scores underspecification 0-3 and unfair-tests 0-3; discard if
either >= 2
```

D. Grading (SWE-bench Resolved rule) and skill-acquisition efficiency.

```
function GRADE(task, patch):
    container = fresh env at task.base_commit (+ environment_setup_commit)
    if not apply(patch): return RESOLVED=false, reason=APPLY
    apply(task.test_patch); run F2P ∪ P2P with timeout
    return RESOLVED = all(F2P pass) and all(P2P pass)

function EFFICIENCY(cohort):                          # Chollet: GD / (P + E), operationalised
    for task in cohort.heldout:
        E_task = attempts*w1 + tokens*w2 + seconds*w3 + senior_calls*w4   # experience spent
until first verified pass (or cap)
        GD_task = 1 - max_i S(task, solved_i)                              # novelty proxy for
generalisation difficulty
    return mean over resolved tasks of GD_task / E_task, plus resolve@budget curve
```

E. Held-out rotation protocol (design). Each cohort k = 20 dev + 10 held-out tasks mined by MINE_TASKS from the user's repositories, all with `created_at` later than the LoRA cut of month $k-1$. During month k: dev tasks may be attempted freely and their outcomes feed the gate log, skills and LoRA data; held-out tasks are attempted once per weekly benchmark run with at most 3 attempts each (ARC's 3-trial rule), in a container, with the skill library, Notebook and LoRA frozen at the run's version hash, and nothing from the attempt (patch, trace, lesson) is written back. At month end: record held-out efficiency and resolve rate; retire the 10 held-out tasks into the dev pool (they may now train the system); mine 10 new held-out tasks and 10 new dev tasks, retiring the 10 oldest dev tasks; run the 0.95-similarity contamination check (RouteLLM) of the new held-out set against the solved-task store and the LoRA training set and replace any flagged task. Report per cohort: resolve rate dev / held-out, efficiency held-out, APGR and CPT(80%) of the gate, and the Eq. 13 similarity of the cohort to the solved store.

```
function ROTATE_COHORT(k):
    heldout_k   -> dev pool (unfreeze: may now enter skills / Notebook / LoRA data)
    heldout_k+1 = MINE_TASKS(repos, since=lora_cut_k)[:10] after triage (discard severity >= 2)
    dev_k+1     = dev pool minus 10 oldest + MINE_TASKS(...)[10:20]
    for t in heldout_k+1: assert max_i S(t, solved_i) < 0.95 and t not in lora_train_k, else
replace
    freeze {skills_hash, notebook_hash, lora_version} for every run in month k+1
```

2.2 Equations and quantities the implementation must compute

- $p_{\text{fast}}(x) = P(\text{verified pass via fast path} \mid x)$ — RouteLLM Eq. 1 objective (maximum likelihood on logged outcomes), with Eq. 2 decision at threshold α.
- $\alpha^* = (C_f + \lambda) / (C_s + \lambda)$ — cost-derived threshold (design, from the asymmetric expected-cost argument above).
- Cost $c = $ fraction of tasks sent to the senior (RouteLLM Eq. 4), and seconds $= \sum_i \left(\tilde{c}_{i,2} \| a_i \| + \tilde{c}_{i,1} \| q \| + \tilde{c}_{i,0}\right)$ (FrugalGPT) with per-stage tokens/s and fixed load times.
- Quality r = resolve rate on the benchmark (RouteLLM Eq. 5 with $s \in \{0,1\}$ from GRADE).
- $PGR = \left(r_{\text{gate}} - r_{\text{fast-only}}\right) / \left(r_{\text{slow-only}} - r_{\text{fast-only}}\right)$; $APGR \approx \frac{1}{10}\sum_{i=1}^{10} PGR(\alpha_i)$; $CPT(x\%) = \min\{c : PGR \geq x\%\}$ (Eqs. 6–8).
- Similarity features: $S(x, x_i)$ cosine over a local embedding, normalised by the nearest-neighbour similarity of x_i (RouteLLM SW ranking); benchmark-vs-solved similarity $\frac{1}{|B|}\sum_{q \in B} \max_i S(q, x_i)$ (RouteLLM Eq. 13) reported per cohort; contamination flag at $S \geq 0.95$.
- FrugalGPT stop rule: return at the first stage i with $g(q, a_i) \geq \tau_i$.
- Efficiency $I = \text{Avg}_T[\omega_T \theta_T \cdot GD / (P + E)]$ (Chollet) with P held fixed within a cohort and the proxies in §2.1 D.

2.3 Data structures and file formats (as TypeScript-style type sketches)

```
        // --- Gate feature vector (design) ----------------------------------------
        type GateFeatures = {
          // similarity to solved tasks (RouteLLM SW ranking / Eq. 13)
          sim_max: number;              // max cosine similarity to any solved task embedding
          sim_knn_success: number;      // gamma^(1+S)-weighted success rate of the k=20 nearest
solved tasks
          skill_match: number;          // best skill-library contract match score in [0,1]; 0
if none
          // structural footprint (SWE-bench Table 1 statistics; SWE-agent behaviour)
          stmt_words: number;           // problem statement length (SWE-bench mean 195.1)
          has_failing_test: boolean;    // an F2P-style test or stack trace is present
          has_stacktrace: boolean;
          est_files: number;            // files named in statement + top retrieval hits above a
score cut
          est_lines: number;            // predicted edit size (gold mean 32.8 lines, 1.7 files)
          retrieval_spread: number;     // entropy of retrieval scores over candidate files
(localisation difficulty)
          repo_loc: number;             // non-test lines in repo (SWE-bench mean 438K)
          cross_module: boolean;        // top-2 candidate files live in different packages
          // self-estimate (RouteLLM causal-LLM classifier style)
          worker_self_p: number;        // worker's P("yes") to "can you solve this without
help?" (logit-derived)
          // history
          fast_fail_ewma: number;       // EWMA of fast-path failures over the last 20 tasks
          loaded_lora: string;          // LoRA pocket id active at decision time (declared
priors)
        };

        // --- Logging schema (design) ---------------------------------------------
        type RouteDecision = {
          task_id: string; ts: string; features: GateFeatures; p_fast: number;
          alpha: number; alpha_version: string; decision: "FAST" | "SLOW"; forced_by_user?:
boolean;
        };
        type StageOutcome = {
          task_id: string; stage: "SKILL" | "WORKER" | "SENIOR"; attempt: number;
          g: number;                    // verifier score; 1.0 = all tests pass
          verifier: ("tests" | "lint" | "typecheck" | "learned")[];
          f2p_passed: number; f2p_total: number; p2p_passed: number; p2p_total: number;
          seconds: number; prompt_tokens: number; output_tokens: number;
          turns: number; failed_edits: number; search_calls: number; files_opened: number;
          patch_lines: number; patch_files: number; accepted: boolean; stop_reason: "pass" |
"threshold" | "budget" | "escalated";
        };
        type TaskOutcome = {
          task_id: string; resolved: boolean; final_stage: "SKILL" | "WORKER" | "SENIOR" |
"USER";
          total_seconds: number; senior_calls: number; label_for_gate: 0 | 1;   // 1 iff FAST
stages resolved it
        };

        // --- Benchmark instance (SWE-bench schema, HF dataset card) ---------------
        type BenchmarkTask = {
          instance_id: string; repo: string; base_commit: string; environment_setup_commit:
string;
          version: string; created_at: string;
          problem_statement: string; hints_text: string;
          patch: string; test_patch: string;             // gold, kept out of the model's reach
          FAIL_TO_PASS: string[]; PASS_TO_PASS: string[];
          triage: { underspecified: 0|1|2|3; unfair_tests: 0|1|2|3; est_minutes: number };  //
SWE-bench Verified rubric
          cohort: string; role: "dev" | "heldout"; mined_after_lora: string;  // LoRA version cut
it postdates
        };
        type BenchmarkRun = {
```

```
          run_id: string; cohort: string; ts: string; model_versions: { worker: string; senior:
string; lora: string; skills_hash: string };
          results: { instance_id: string; resolved: boolean; apply_ok: boolean; stage: string; E:
number; GD: number }[];
          resolve_rate_dev: number; resolve_rate_heldout: number; efficiency_heldout: number;
          cpt80: number; apgr: number; contamination_flags: string[];
        };
```

On disk: one JSONL per log type (`routes.jsonl`, `stages.jsonl`, `outcomes.jsonl`), benchmark tasks as `bench/<cohort>/<instance_id>.json`, runs as `bench/runs/<run_id>.json`, and a `predictions.jsonl` of `{instance_id, model_patch, model_name_or_path}` for the grader (SWE-bench harness convention).

2.4 Hyperparameters and defaults to start from (with the paper each comes from)

Quantity	Default	Source
Neighbour weighting γ	10	RouteLLM SW ranking
Contamination similarity cut	0.95 cosine	RouteLLM contamination check
APGR sweep points	10 call-fraction points	RouteLLM Eq. 8
Gate target	report CPT(50%), CPT(80%), APGR	RouteLLM §3.2
Gate learner (if >50 outcomes)	logistic head; Adam lr 3e-4, wd 1e-5, batch 64, ~10 epochs if an MF-style embedding head is used	RouteLLM MF settings
Cascade length	3 (skill → worker → senior)	FrugalGPT (cascade length 3)
Per-stage thresholds τ	1.0 when tests exist (all F2P+P2P pass); start 0.9 / 0.5 for learned g at skill / worker	FrugalGPT HEADLINES thresholds 0.96 / 0.37 (shape only)
Scorer g without tests	small classifier (DistilBERT-size) on (task, patch) → pass	FrugalGPT
Search result cap	50	SWE-agent
Viewer window / scroll	100 lines	SWE-agent (30 → -3.7 pts, full file → -5.3 pts)
History kept in full	last 5 observations	SWE-agent (full history → -3.0 pts)
Edit guardrail	lint-reject on syntax error	SWE-agent (w/o linting → -3.0 pts)
Early-escalation trigger	2nd failed edit	SWE-agent (90.5% → 57.2% success after one failure)
Per-task budget cap	time-equivalent of ~30–40 turns; auto-submit on exhaustion	SWE-agent $4 cap
Sampling temperature (worker)	0.0	SWE-agent
Oracle-collapsed context buffer	±15 lines around edited regions	SWE-bench
Context budget warning	resolve rate drops with context; cap retrieved context at the 13k-token tier first	SWE-bench Table 3 / Fig. 5
Task triage discard rule	severity ≥2 on underspecification or unfair tests	SWE-bench Verified
Benchmark size	20 dev + 10 held-out, 3 attempts max per held-out task	task spec; ARC's 3 trials
Held-out rotation	monthly	task spec; SWE-bench temporal split

2.5 Evaluation: how to know the module works (metrics, from the papers)

1. **Gate quality (RouteLLM).** On the 20 dev tasks run three configurations — fast-only, slow-only, gated at the 10 α_i — and compute r, c, PGR, APGR, CPT(50%), CPT(80%). The gate works if APGR > 0.5 (better than random) and CPT(80%) is well below 80% of senior calls; RouteLLM's augmented routers reach APGR 0.60–0.80 and CPT(80%) 31–72% as reference points. Also report the Eq. 13 similarity between the cohort and the solved-task store — a low value predicts a weak gate (RouteLLM Table 4).
2. **Cascade cost (FrugalGPT).** Plot resolve rate vs seconds per task for the cascade and for senior-only; the cascade must match senior-only resolve rate at a fraction of the cost (FrugalGPT: 59–98% cost savings at equal accuracy). Track the FrugalGPT failure case: tasks where every stage ran and agreed.
3. **Benchmark (SWE-bench / SWE-agent).** % Resolved and % Apply on dev and on held-out, graded in a container by the Resolved rule; report both because apply failures are a distinct failure mode (SWE-bench apply rates 26–54% with resolve rates 0.2–3.8%). Also log turns, failed edits and submission-before-budget rate (SWE-agent: 93.0% for resolved vs 69.0% overall).
4. **Learning (Chollet).** The month-over-month quantity is efficiency on held-out tasks (GD/E proxy) and resolve@budget curves — not dev-set resolve rate, which can rise through leakage into skills/LoRA. A system that is learning shows rising held-out efficiency with priors (base model) fixed; a system that is only memorising shows rising dev rate and flat held-out efficiency.
5. **Calibration.** Reliability diagram of p_{fast} vs empirical fast-path success per decile; expected calibration error after isotonic fit.

2.6 Known pitfalls and failure modes (from the papers' limitations)

- Routers trained on one distribution are near random on another (RouteLLM: Arena-only MMLU APGR 0.47–0.52); the gate needs logged outcomes *from the user's own repos*, and a small in-domain set moves it most (1,500 golden samples, <2% of data, gave +20.7% APGR).
- Router behaviour varies widely between architectures on identical data (RouteLLM) — keep the gate simple (SW ranking / logistic) and compare against the random baseline every month.
- Judge-based labels are biased (RouteLLM); prefer execution outcomes as labels, and treat learned g as a fallback only.
- Cascades waste calls when stages agree but the scorer cannot tell correctness (FrugalGPT open problem); with execution verifiers this collapses to "no tests available", so the mined benchmark should prefer tasks with F2P tests.
- Test-based grading can reject valid but different solutions and accept narrow ones (SWE-bench; Verified found 61.1% of instances with potentially unfair tests and 38.3% underspecified) — triage mined tasks with the 0–3 rubrics and discard at ≥2.
- More context lowers resolve rate even as recall rises (SWE-bench) — the footprint features must penalise retrieval spread, and the worker should get the smallest sufficient context.
- Agents repeat failed edits and use late turns poorly (SWE-agent) — escalate on the second failed edit; cap turns.
- Memorisation masquerades as learning (SWE-bench temporal split; Chollet developer-aware generalisation) — held-out tasks must postdate the last LoRA cut and never enter skills, Notebook or training data; check contamination at 0.95 similarity.
- Generated patches are shorter and single-file compared with gold (SWE-bench Table 8) — a high `est_files` is a strong slow-path signal.

2.7 Open questions this section does not settle

- How to map α to a seconds budget when C_s varies with MoE expert offload load (RouteLLM gives only a sweep, not a closed form).

- Whether the worker's self-estimate is calibrated enough to be a feature at 9B scale; RouteLLM's causal-LLM router needed full fine-tuning of an 8B model on 65K+ labels.
- The exact SWE-Llama LoRA hyperparameters and the SWE-bench Lite criteria (Appendix B / A.7 not in fetched text) — relevant if the benchmark is also used to produce weekly training pairs.
- Chollet's GD, P, E are uncomputable; the proxies here (novelty distance, token/attempt counts) are design choices to be validated against the user's judgement of task difficulty.
- Monthly held-out rotation with 10 tasks gives wide confidence intervals (RouteLLM's random baseline shows ±4–5 points on 160 items); the protocol should accumulate cohorts and report rolling 3-month efficiency.
- Multi-stage routing beyond two paths (skill vs worker vs senior as a 3-way gate) is explicitly future work in RouteLLM.

3. Verified source table

arXiv id	Title	Year	Venue	Code	PDF
2406.18665	RouteLLM: Learning to Route LLMs with Preference Data	2024 (v4 2025)	arXiv; venue not on abs page (ICLR 2025 — verify)	https://github.com/lm-sys/RouteLLM	https://arxiv.org/pdf/2406.18665
2305.05176	FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance	2023	arXiv; venue not on abs page (TMLR 2024 — verify)	https://github.com/stanford-futuredata/FrugalGPT	https://arxiv.org/pdf/2305.05176
2310.06770	SWE-bench: Can Language Models Resolve Real-World GitHub Issues?	2024	ICLR 2024	https://github.com/SWE-bench/SWE-bench	https://arxiv.org/pdf/2310.06770
1911.01547	On the Measure of Intelligence	2019	arXiv (no venue)	https://github.com/fchollet/ARC-AGI	https://arxiv.org/pdf/1911.01547
2405.15793	SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering	2024	arXiv; venue not on abs page (NeurIPS 2024 — verify)	https://github.com/SWE-agent/SWE-agent	https://arxiv.org/pdf/2405.15793